TRIRACH Adam

Projet 1 : Répertoire téléphonique (Web et BD)

Partie Développement

Choix du langage et de la technologie

Le projet a été développé avec Python et le micro-framework Flask, qui permet de créer facilement des sites web dynamiques. Flask offre une architecture simple basée sur trois éléments principaux :

- Les routes (définies avec @app.route) qui relient les pages web aux fonctions Python.
- Les templates HTML (dans le dossier templates/) qui contiennent l'affichage du site.
- Une base de données SQLite utilisée pour stocker les utilisateurs et leurs informations.

Structure de la base de données

Le site s'appuie sur une base de données SQLite, simple à utiliser et intégrée à Python sans serveur externe. Deux tables principales sont nécessaires :

Table UTILISATEUR

Champ	Type	Description
`id_user`	INTEGER	Identifiant unique de l'utilisateur
`username`	TEXT	Identifiant de connexion
`password_hash`	TEXT	Mot de passe chiffré (hashé)
`id_contact`	INTEGER	Lien vers la fiche contact correspondante

La table UTILISATEUR contient une clé étrangère id_contact qui référence la table CONTACT.
Cela permet de lier chaque compte utilisateur à sa fiche personnelle dans le répertoire téléphonique.
Ce lien est essentiel pour que le site sache quelle fiche afficher ou modifier après la connexion.

Table CONTACT

Champ	Type	Description
`id_contact`	INTEGER	Identifiant unique de la fiche
`nom`	TEXT	Nom du contact
`prenom`	TEXT	Prénom du contact
`date_naissance`	TEXT	Date de naissance
`adresse`	TEXT	Adresse postale
`profession`	TEXT	Profession
`phone_fixe`	TEXT	Numéro de téléphone fixe
`phone_portable`	TEXT	Numéro de téléphone portable
`email`	TEXT	Adresse mail

MLD de la base de donnée

UTILISATEUR(id_user, username, password_hash, id_contact)

CONTACT(id_contact, nom, prenom, date_naissance, adresse, profession, phone_fixe, phone_portable, email)

Fonctionnement de Flask

Flask est une bibliothèque Python qui permet de créer facilement des sites web dynamiques. Elle repose sur le principe suivant : lorsqu'un utilisateur accède à une page du site, Flask reçoit la requête, exécute la fonction Python correspondante et renvoie une page HTML. Cette page est générée à partir d'un modèle (template) dans lequel on peut insérer des variables, des conditions et des boucles Python grâce à un langage appelé Jinja2.

Le fonctionnement général d'un site Flask repose sur trois éléments :

- un fichier principal en Python (souvent nommé app.py) qui contient le code du serveur et la logique du site
- des fichiers HTML dans un dossier appelé "templates" pour les pages web
- parfois des fichiers CSS dans un dossier "static"

Lorsqu'une personne visite une adresse du site, Flask fait correspondre cette adresse à une fonction Python appelée "route". Chaque route est définie par un décorateur de la forme :

```
@app.route("/nom_de_page")
def fonction():
return render_template("page.html")
```

Ainsi, la fonction est exécutée lorsque l'utilisateur se rend à l'adresse indiquée, et la page HTML spécifiée est affichée. On peut aussi préciser les méthodes HTTP autorisées, par exemple "GET" pour afficher une page et "POST" pour envoyer un formulaire :

```
@app.route("/login", methods=["GET", "POST"])
```

Le lien entre Python et HTML se fait grâce au moteur de templates Jinja2. Il permet d'insérer des variables et des structures de contrôle directement dans les fichiers HTML. Les variables sont écrites entre doubles accolades, par exemple : {{ username }}. On peut aussi écrire des conditions ou des boucles avec des balises spéciales :

```
{% if user %} Bonjour {{ user }} {% endif %}
{% for contact in contacts %} {{ contact.nom }} {% endfor %}
```

Dans le code Python, plusieurs fonctions de Flask reviennent très souvent :

- Flask(name) : crée l'application web Flask.
- @app.route() : associe une adresse web à une fonction Python.
- render_template("page.html", variable=valeur) : envoie des données Python vers page HTML.
- request : permet de récupérer les données envoyées par l'utilisateur (par exemple dans un formulaire).
- redirect(url_for("nom_fonction")) : redirige l'utilisateur vers une autre page.
- flash("message") : affiche un message temporaire d'information ou d'erreur.
- session : conserve des informations pendant la navigation, par exemple l'identifiant d'un utilisateur connecté.

Les sessions fonctionnent comme un dictionnaire Python et sont protégées par une clé secrète appelée “SECRET_KEY”. Cela permet de reconnaître un utilisateur d’une page à l’autre sans qu’il ait besoin de se reconnecter.

Dans les sites qui nécessitent une authentification, on utilise aussi la bibliothèque “werkzeug.security” incluse avec Flask pour sécuriser les mots de passe. Deux fonctions principales sont utilisées :

- `generate_password_hash(mdp)` : transforme un mot de passe en une version chiffrée (appelée hash).
- `check_password_hash(hash, mdp)` : vérifie qu’un mot de passe correspond bien au hash enregistré.

Ainsi, les mots de passe ne sont jamais stockés en clair dans la base de données, ce qui améliore la sécurité.

Bibliothèque « os » :

La bibliothèque « os » (Operating System) une bibliothèque standard de Python qui permet d’interagir avec le système d’exploitation. Elle sert notamment à manipuler les fichiers, les dossiers et les chemins d’accès.

Dans notre projet Flask, elle peut être utilisée pour vérifier si la base de données existe déjà (os.path.exists), pour créer un dossier ou pour construire des chemins d’accès indépendants du système.

Bibliothèque SQLite3 :

La bibliothèque sqlite3 est une bibliothèque standard de Python qui permet d’interagir facilement avec une base de données relationnelle au format SQLite.
SQLite est un système de gestion de base de données léger et intégré, qui ne nécessite ni installation ni serveur externe : toutes les données sont stockées dans un simple fichier (ici repertoire.db).
Dans ce projet, sqlite3 est utilisée pour :

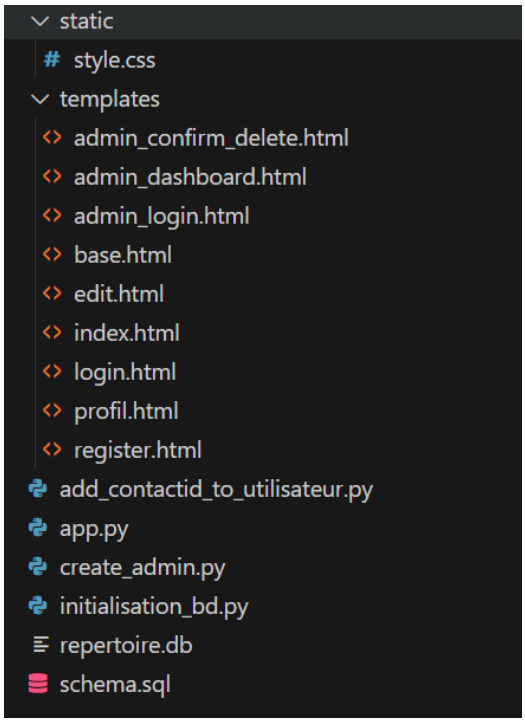
- Créer la base de données à partir du fichier schema.sql (grâce à la méthode executescript()).
- Se connecter à la base (sqlite3.connect("repertoire.db")).
- Exécuter des requêtes SQL (SELECT, INSERT, UPDATE, DELETE) à partir du code Python.
- Lire les résultats des requêtes avec fetchone() ou fetchall().

Dans le projet, chaque fonction qui accède à la base crée une connexion grâce à la fonction personnalisée get_conn().
Cette fonction configure aussi row_factory = sqlite3.Row, ce qui permet de lire les colonnes par leur nom plutôt que par leur position (par exemple contact["nom"] au lieu de contact[1]).

Les requêtes SQL sont paramétrées (avec ? et un tuple de valeurs) pour éviter les injections SQL, une faille de sécurité courante :

```
conn.execute("SELECT * FROM CONTACT WHERE nom = ?", (nom,))
```

Arborescence du projet :



Le fonctionnement de chaque page est commenté dans « app.py ». Le reste du code est aussi commenté.

Remarque : Le fichier `create_admin.py` avait pour but de créer un compte administrateur directement dans la base, mais cette étape a été remplacée par une gestion interne dans le code Python (app.py). Il n'est donc plus nécessaire de l'utiliser.

Test du site

Préparatifs

Pour tester le site, il suffit d'avoir installé Python (version 3.10 ou plus récente). Si ce n'est pas déjà fait, installer la bibliothèque Flask sur VS Code ou en ouvrant le terminal sur le dossier. Entrer :

```
pip install flask
```

On n'aura pas besoin d'installer « werkzeug » car il est déjà inclus dans Flask. Et « os » est déjà inclus dans la bibliothèque standard de python, tout comme « sqlite3 ».

Pour manipuler le code, utiliser VS Code et ouvrir le dossier du projet (ctrl K + Crtl O). Vous pourrez ensuite naviguer entre les fichiers du projet.

Vérifier que les fichiers suivants sont bien présents :

- app.py
- initialisation_bd.py
- schema.sql

- le dossier templates/ (avec toutes les pages HTML)
- le dossier static/ (avec style.css)

Ouvrir un terminal ou un invite de commandes dans le dossier du projet.

Avant le premier lancement, exécuter le fichier initialisation_bd.py pour (re)créer la base de données.
Commande à exécuter :

```
python initialisation_bd.py
```

Ce script crée le fichier « repertoire.db » à partir de « schema.sql ».
Si le fichier existe déjà, il sera supprimé et remplacé.
Un message « Info : BD créée avec succès » doit s'afficher.

Lancement du site :

Une fois la base prête, le site peut être lancé avec :
python app.py

Flask démarre alors un petit serveur local.
Le terminal affichera une ligne du type :

- Running on http://127.0.0.1:5000/

Il suffit ensuite d'ouvrir un navigateur web et de taper cette adresse (http://127.0.0.1:5000/) pour accéder à la page d'accueil.

Test des fonctionnalités principales

Page d'accueil

- Affiche la liste de tous les contacts du répertoire.
- Tester la recherche : entrer un nom, une profession ou un email dans les champs et vérifier que les résultats s'actualisent.
- Si aucun contact ne correspond, la liste doit simplement être vide.

Inscription

- Cliquer sur « Inscription » pour créer un nouveau compte utilisateur.
- Remplir tous les champs demandés (nom, prénom, date de naissance, adresse, etc.).
- Après validation, un message doit s'afficher : « Compte et fiche créés avec succès. »
- Le nouvel utilisateur est désormais visible dans la base de données.

Connexion

- Après l'inscription, vous serez automatiquement dirigé vers la page de connexion.
- Entrer le nom d'utilisateur et le mot de passe choisis.
- Si la connexion réussit, l'utilisateur est redirigé vers sa page « Profil ».
- En cas d'erreur, un message rouge indique que l'identifiant ou le mot de passe est incorrect.
- Si vous voulez vous connecter dès l'entrée du site, cliquer sur « Inscription » et ensuite sur « Déjà un compte ? ».

Modification du profil

- Depuis la page « Profil », modifier une information (par exemple la profession).
- Cliquer sur « Enregistrer » : un message « Fiche mise à jour » doit apparaître.
- Retourner à l'accueil pour vérifier que la modification est bien visible dans la liste.
- Vous pouvez supprimer votre compte en cliquant sur « Suppr. », une page de confirmation de suppression apparaîtra. Il faudra se déconnecter immédiatement après cette action.

Déconnexion

- Cliquer sur « Déconnexion » pour quitter le compte.
- L'utilisateur doit être renvoyé vers la page d'accueil sans possibilité d'accéder à son profil sans se reconnecter.

Connexion admin

- Cliquer sur « Connexion Admin » ou accéder directement à /admin_login
- Se connecter avec :
  `identifiant : admin`
  `mot de passe : 123456`
- L'administrateur accède à un tableau de bord affichant toutes les fiches du répertoire.
- Il peut modifier ou supprimer n'importe quelle fiche.
- En cas de suppression, une page de confirmation s'affiche avant la validation.
- La déconnexion se fait via « Déconnexion Admin ».

En cas d'erreur…

Plusieurs cas peuvent se présenter :

- Si un message d'erreur indique « `no such table: CONTACT` » :
  Cela signifie que la base de données n'a pas été créée correctement.
  Solution : relancer le script `initialisation_bd.py` puis redémarrer Flask.

- Si le serveur Flask ne démarre pas (erreur « module flask introuvable ») :
  Installer Flask avec la commande :
  `pip install flask`

- Si un message d'erreur apparaît sur le site :
  Lire le message dans le terminal (Flask indique la ligne concernée).
  Dans la plupart des cas, il s'agit d'une faute de frappe dans un champ ou d'un oubli de base de données.

À la fin du test

Il est conseillé de retourner dans le terminal où « app.py » a été lancé et appuyer sur « Ctrl + C » pour « fermer » le site.